

Logging – Log4j



Contents

- What is Logging?
- What is Log4j in Selenium?
- Components of Log4j
- How to Set up Log4j in Selenium
- How to Use Log4j in Selenium
- Log4j with Properties File Configuration
- Why Use Log4j in Selenium?

Logging

- Logging is a process that takes applications to a newer level with information logs on how the applications may or may not have performed/executed. It gives an exact idea of software performance, including any shortcomings.
- Log4j in Selenium is one such logging framework that helps in gathering information in the form of logs or log files.

Log4j

- Log4j is a logging framework written in Java that provides an easy way for logging in Selenium.
- In a nutshell, the framework gives out information about everything that goes on during the software execution.
- Log4j also provides insight into anything that may have gone wrong during software execution or automation.
- Overall, Log4j documents the output in the form of logs that can be examined later for purposes such as auditing small and large-scale Selenium projects.

Log4j Components

The Log4j logging framework comprises the following components:

1. Logger
2. Appenders
3. Layout

1. Logger

The function of the logger in Log4j is basically storing and capturing all the necessary logging information that will be generated using the framework.

- **Logger Class** – To fully use the logger, **create an instance for a logger class where all the generic methods will be at the user's disposal**, required to use Log4j.
- **Log Levels** – These are the methods that will be used to print the log messages. There are primarily only a few log levels that are used in a script.

Logging – Log4j

Logger levels -Log4j

1. **ALL** – This level will prioritize and include everything in the logs.
2. **ERROR** – This level will show messages that inform users about error events that may not stop the application.
3. **WARN** – This level will show information regarding warnings, that may not stop the execution but may still cause problems.
4. **DEBUG** – This level will log debugging information.
5. **INFO** – This level will log the progress of the application.
6. **FATAL** – This will print information critical to the system that may even crash the application.

Logger levels -Log4j2

OFF

FATAL

ERROR

WARN

INFO

DEBUG

TRACE

ALL

To start logging messages using this basic configuration, all you need to do is obtain a Logger instance using the LogManager class:

```
private static Logger logger = LogManager.getLogger(MyService.class);
```

Then you can use the logger object with methods corresponding to the log level you want:

```
logger.error("This is an error message");
```

Syntax

Appenders

- The appender basically grabs information from the logger and writes log messages to a file or any other storage location.
- The following are some of the appenders one can use for Log4j:
 - 1. FileAppender** – This will append the log messages to a file.
 - 2. RollingFileAppender** – It will perform the same function as FileAppender, but users will be able to specify the maximum file size. Once the limit is exceeded, the appender will create another file to write the messages.
 - 3. DailyRollingFileAppender** – It specifies the frequency by which the messages are to be written to the file.
 - 4. ConsoleAppender** – In this, the appender will simply write the log messages in the console.

Layout

- The layout is where the format in which log messages will appear is decided.
- There are several layouts one can use for log messages:
 - 1. Pattern Layout** – The user must specify a conversion pattern based on which the logs will be displayed. Otherwise, it takes the default conversion pattern in case of “no pattern specified”.
 - 2. HTML Layout** – In this layout, the format of logs will be in the form of an HTML table.
 - 3. XML Layout** – This will show the logs in an XML format

Dependency to be added in pom.xml -log4j

```
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.19.0</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-api -->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.19.0</version>
</dependency>
```

Logging Execution steps

Follow the steps below to successfully run Log4j with Selenium Webdriver:

1. Write an automation script
2. After creating the script, create a log4j.properties file and specify the root logger, appended, and layout information in the file.
3. Import log4j dependencies like Logger, PropertyConfigurator, and add them to the script along with the logger class.
4. Add the messages that will be displayed in the log file

Logging – Log4j

log4j2.properties file

- **Create log4j2.properties file i n src/main/resources/log4j2.properties**

Configuration name

name = PropertiesConfig

Log file output folder

property.filename = logs

Define appenders

appenders = console, file

Console Appender

appender.console.type = Console

appender.console.name = STDOUT

appender.console.layout.type = PatternLayout

appender.console.layout.pattern = [%-5level] %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %c{1} - %msg%n

log4j2.properties file

- **Create log4j2.properties file in src/main/resources/log4j2.properties**

```
# File Appender
appender.file.type = File
appender.file.name = LOGFILE
appender.file.fileName = ${filename}/logs.log
appender.file.layout.type = PatternLayout
appender.file.layout.pattern = [%-5level] %d{yyyy-MM-dd HH:mm:ss.SSS} [%t] %c{1} - %msg%n

# Root Logger Configuration
rootLogger.level = debug
rootLogger.appenderRefs = stdout, file
rootLogger.appenderRef.stdout.ref = STDOUT
rootLogger.appenderRef.file.ref = LOGFILE
```

tests/OrangeHRMLoginTest.java - updated with log4j

```
package tests;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import pages.LoginPage;
import pages.HomePage;
import utils.CSVUtils;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import java.util.List;

public class OrangeHRMLoginTest {
    public static Logger log = LogManager.getLogger(OrangeHRMLoginTest.class);

    public static void main(String[] args) {
        WebDriverManager.chromedriver().setup();

        List<String[]> testData = CSVUtils.readCSV("testdata/loginData.csv");
```

tests/OrangeHRMLoginTest.java - updated with log4j

```
for (String[] row : testData) {
    String username = row[0];
    String password = row[1];
    runLoginTest(username, password);
}

public static void runLoginTest(String username, String password) {
    WebDriver driver = new ChromeDriver();
    driver.manage().window().maximize();
    log.info("Launching browser and navigating to OrangeHRM");
    driver.get("https://opensource-demo.orangehrmlive.com/");

    try {
        LoginPage loginPage = new LoginPage(driver);
        log.info("Attempting login with username: " + username);
        loginPage.login(username, password);

        Thread.sleep(2000); // Wait for page load (demo purpose)
    }
}
```

tests/OrangeHRMLoginTest.java - updated with log4j

```
HomePage homePage = new HomePage(driver);

if (homePage.isDashboardDisplayed()) {
    System.out.println("Login successful for user: " + username + "\n");
    log.info("Login successful for user: " + username);
} else {
    System.out.println("Login failed for user: " + username + " - Dashboard not found.\n");
    log.warn("Login failed for user: " + username + " - Dashboard not found");
}

} catch (Exception e) {
    System.out.println("Exception occurred while testing user: " + username + "\n");
    log.error("Exception occurred for user: " + username + " - " + e.getMessage());
    e.printStackTrace();
} finally {
    driver.quit();
    log.info("Browser closed for user: " + username);
}

}
```

tests/OrangeHRMLoginTest.java

Output in Console

```
12:01:35.432 [main] INFO tests.OrangeHRMLoginTest - Launching browser and navigating to OrangeHRM
12:01:37.145 [main] INFO tests.OrangeHRMLoginTest - Attempting login with username: Admin
Login successful for user: Admin
```

```
12:01:41.785 [main] INFO tests.OrangeHRMLoginTest - Browser closed for user: Admin
```

```
12:01:42.012 [main] INFO tests.OrangeHRMLoginTest - Launching browser and navigating to OrangeHRM
12:01:43.751 [main] INFO tests.OrangeHRMLoginTest - Attempting login with username: wronguser
Login failed for user: wronguser - Dashboard not found.
```

```
12:01:47.432 [main] WARN tests.OrangeHRMLoginTest - Login failed for user: wronguser - Dashboard not found
12:01:47.567 [main] INFO tests.OrangeHRMLoginTest - Browser closed for user: wronguser
```

tests/OrangeHRMLoginTest.java

logs/logs.log (File Log Output)

```
[INFO ] 2025-08-04 12:01:35.432 [main] OrangeHRMLoginTest - Launching browser and navigating to Ora
[INFO ] 2025-08-04 12:01:37.145 [main] OrangeHRMLoginTest - Attempting login with username: Admin
[INFO ] 2025-08-04 12:01:41.785 [main] OrangeHRMLoginTest - Login successful for user: Admin
[INFO ] 2025-08-04 12:01:41.789 [main] OrangeHRMLoginTest - Browser closed for user: Admin

[INFO ] 2025-08-04 12:01:42.012 [main] OrangeHRMLoginTest - Launching browser and navigating to Ora
[INFO ] 2025-08-04 12:01:43.751 [main] OrangeHRMLoginTest - Attempting login with username: wrongus
[WARN ] 2025-08-04 12:01:47.432 [main] OrangeHRMLoginTest - Login failed for user: wronguser - Dash
[INFO ] 2025-08-04 12:01:47.567 [main] OrangeHRMLoginTest - Browser closed for user: wronguser
```



Quiz



1) Which of the following is NOT a valid Log4j logging level?

a) INFO

b) DEBUG

c) WARN

d) PRINT

Answer : Option d)

Quiz



2) What is the correct method to create a Log4j Logger instance?

a) `Logger log = new Logger();`

b) `Logger log = Logger.getLogger(Logger.class);`

c) `Logger log = Logger.create();`

d) `Logger log = LogManager.get();`

Answer : Option b)

Quiz



3) In a log4j.properties file, what does this line do?

log4j.rootLogger=DEBUG, file

a) Sets file name to DEBUG

b) Logs only file-level logs

c) Sets root log level to DEBUG and output to a file

d) Disables console logging

Answer : Option c)

Quiz



4) What is an Appender in Log4j?

a) A method to close logging

b) A tool to modify logs

c) A component that defines where log messages go (e.g., console, file)

d) A formatter for test steps

Answer : Option c)

Quiz



5) Which of the following layouts is commonly used in Log4j for readable output?

a) Simple Layout

b) HTML Layout

c) Pattern Layout

d) All the above

Answer : Option d)

THANK YOU